Agent Unit Economics, Worked Out by Hand

Why your true cost per task is the cost per *successful* task —
and how stacked levers multiply, not add.

What this document does. It starts from one mean trajectory cost ($0.08) and one success rate (0.85) and works out the number that actually governs your margin: cost per *successful* task. It then adds retries, derives gross margin and break-even volume, and shows how the cost levers from the module combine multiplicatively. Every figure is reproduced by the program in Appendix A.

Where this sits. This is **Topic 2 of Module 3.4** (cost & latency profiling, unit economics). Companions: Module 3.4 Topic 1 (the cost tail / p99) and Module 4.3 (cost engineering, where the levers are built).

Concepts covered, in order: mean cost per call · cost per successful task · expected attempts under retry · gross margin · break-even volume · stacked levers as a product.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one cost, one success rate

Quantity	Symbol	Value here
Mean cost per trajectory (call)	c	$0.08
Success rate	s	0.85
Price charged per task	P	$0.25
Fixed monthly cost	F	$1,000
Lever reductions (early-exit, caching)	r_1, r_2	0.30, 0.15

Notation. "Cost per call" c is what you pay every time the agent runs, success or failure. "Cost per successful task" is what you pay per result you can actually bill for — the only cost that matters for pricing.

> **Intuition**
>
> A failed agent run is not free. The tokens were spent, the tools were called, the latency was paid — you simply got nothing sellable out the other end. So the honest denominator is *successful* tasks, not total calls. The lower your success rate, the more every failure quietly taxes every success.

2 Step 1 — cost per successful task

If a fraction s of calls succeed, then producing one success consumes on average $1/s$ calls (you keep paying c until one lands). Hence

$$c_{\text{success}} = \frac{c}{s} = \frac{0.08}{0.85} = \$\mathbf{0.0941}. \tag{1}$$

The expected number of attempts to a first success is $1/s = 1/0.85 = \mathbf{1.176}$ (a geometric distribution: $\mathbb{E}[\text{attempts}] = 1/s$). ✓ A 15% failure rate inflates the real unit cost by 18% — from \$0.080 to \$0.094 — before you have charged anyone a cent.

Mini-example — this operation in isolation

The geometric expectation in one line: $P(\text{first success on attempt } k) = (1-s)^{k-1}s$, and $\sum_{k\geq 1} k(1-s)^{k-1}s = 1/s$. With $s = 0.85$ that is 1.176 attempts. Retrying on failure therefore costs c/s in expectation — exactly Equation (1), which is why "cost with retries" and "cost per success" are the same number.

3 Step 2 — margin and break-even

Charging $P = \$0.25$ against a true unit cost of $c_{\text{success}} = \$0.0941$:

$$\text{gross margin} = \frac{P - c_{\text{success}}}{P} = \frac{0.25 - 0.0941}{0.25} = \mathbf{0.624}\ (62.4\%).$$

(If instead you wrongly used the raw $c = \$0.08$ you would report margin 0.68 — a flattering 4-point overstatement born of ignoring failures.) Break-even volume against the fixed cost F:

$$V^{\star} = \frac{F}{P - c_{\text{success}}} = \frac{1000}{0.25 - 0.09412} = \frac{1000}{0.15588} \approx \mathbf{6{,}415}\ \text{tasks/month}. \tag{2}$$

Below ∼6.4k successful tasks you lose money; above it you clear the fixed cost. Using the naive cost would have told you $1000/0.17 = 5{,}882$ — you would have planned to break even ∼530 tasks too early.

4 Step 3 — levers multiply

The module lists cost levers (routing, caching, pruning, early-exit, …). Each removes a *fraction* of cost, so stacking two does not add their savings — it multiplies what remains. With reductions $r_1 = 0.30$ (early-exit) and $r_2 = 0.15$ (caching):

$$\boxed{\text{combined factor} = \prod_i (1 - r_i) = (1 - 0.30)(1 - 0.15) = 0.70 \cdot 0.85 = 0.595.} \tag{3}$$

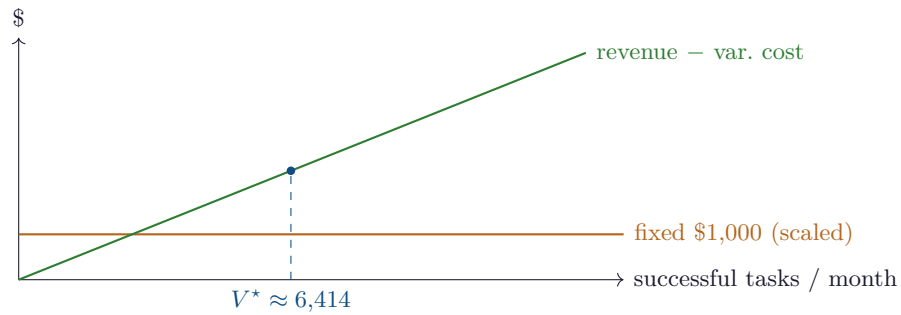
So two levers cut cost to 59.5% of baseline — a **40.5**% reduction, *not* $30 + 15 = 45\%$. Applied to c_{success}: $0.0941 \times 0.595 = \$0.0560$ per success, lifting gross margin from 62.4% to 77.6%.

Scenario	cost/success	margin at \$0.25	break-even (F=\$1k)
raw cost (ignores failures)	\$0.0800	68.0%	5,882
true (cost/success)	\$0.0941	62.4%	6,415
+ two levers (×0.595)	\$0.0560	77.6%	5,155

Watch out

Lever reductions are only multiplicative when they act on *different* cost components. Two levers that both shrink the same tokens overlap and you cannot just multiply — measure the combined effect end-to-end. And no lever helps a failure: raising the success rate s shifts the whole curve, because it sits in the denominator of every figure above.

5 The economics, drawn



The contribution line $(P - c_{\text{success}}) \times V$ crosses the fixed-cost line at $V^\star$. Lowering c_{success} (levers, higher s) steepens the line and pulls break-even left.

6 Cost cheat-sheet

Cost per success	$c_{\text{success}} = c/s$
Expected attempts	$1/s$ (geometric)
Gross margin	$(P - c_{\text{success}})/P$
Break-even volume	$V^\star = F/(P - c_{\text{success}})$
Stacked levers	factor $= \prod_i (1 - r_i)$
Reduction	$1 - \prod_i (1 - r_i)$

7 An honest word about these numbers

The $0.08 cost, 0.85 success rate, $0.25 price and $1,000 fixed cost are illustrative round inputs. Your real c comes from the cost tail (Topic 1), your s from the evaluation harness, and your levers' reductions must be measured, not assumed. What is invariant: divide by the success rate (failures are real spend), and combine fractional savings by multiplying $(1 - r_i)$, never by adding. Optimise cost-per-success, and the margin follows.

Appendix A — Reference implementation

```
# unit_economics.py -- reproduces every number in "Agent Unit Economics".
c, s, P, F = 0.08, 0.85, 0.25, 1000.0

```

```python
c_success = c / s
attempts = 1 / s
margin = (P - c_success) / P
breakeven = F / (P - c_success)
print(f"cost/success = ${c_success:.4f} expected attempts = {attempts:.3f}")
print(f"gross margin = {margin:.3f} break-even = {breakeven:.0f} tasks")

factor = (1 - 0.30) * (1 - 0.15) # stacked levers multiply
print(f"two-lever factor = {factor:.3f} reduction = {(1-factor)*100:.1f}%")
c2 = c_success * factor
print(f"after levers: cost/success = ${c2:.4f} margin = {(P-c2)/P:.3f} "
      f"break-even = {F/(P-c2):.0f}")

# Expected output:
# cost/success = $0.0941 expected attempts = 1.176
# gross margin = 0.624 break-even = 6415 tasks
# two-lever factor = 0.595 reduction = 40.5%
# after levers: cost/success = $0.0560 margin = 0.776 break-even = 5155
```

— end of document —